

Exercises

Exercise 2.1

Question 2.1.1

Show mathematically that the marginal distribution on return values for these three programs is the same by directly computing the probability using the rules of probability.

(*Hint*: write down each possible history of random choices for each program).

```
flip() ? flip(.7) : flip(.1)
```

```
flip(flip() ? .7 : .1)
```

```
flip(.4)
```

Question 2.2.2

Check your answers by sampling from the programs, 1000 times each, and plotting the results.

Question 2.2.3

Write another new program with the same marginal distribution on return values that looks different from all of the above programs.

Exercise 2.2

Question 2.2.1

Explain why (in terms of the evaluation process) these two programs give different answers (i.e., why they have different distributions on return values).

```
var foo = flip()
display([foo, foo, foo])
```

```
var foo = function() {
  return flip()
}
display([foo(), foo(), foo()])
```

Question 2.2.2

Modify the second program using `mem` so that it has the same distribution as the first program.

Question 2.2.3

Change the program in Question 2.2.2 so that the first two elements in the list are always the same as each other, but the third element can be different.

(*Optional challenge:* try to do this by adding only these 4 characters: `x`, `0`, `0`, and `1`.)

Exercise 2.3

Question 2.3.1

Which of these programs would be more likely to generate the following proportions for 100 values of `C`? Justify your response.

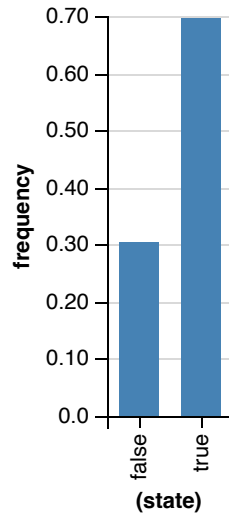


Figure 1: Histogram for Q2.3.1

```
// Program "A"
var A = flip()
var B = flip(0.9)
var C = flip()
  ? A && B
  : A || B
display([A, B, C])
```

```
// Program "B"
var A = flip(0.9);
var B = A && flip(0.9)
var C = B && flip(0.9)
display([A, B, C])
```

Question 2.3.2

Could the program you did *not* choose in Question 2.3.1 have *also* generated those return values? Explain.

Exercise 2.4

In the simple medical diagnosis example, we imagined a generative process for diseases and symptoms of a single patient. In this exercise, we'll write a version of that model that represents the diseases and symptoms of many patients.

Question 2.4.1

Let's look at just two common conditions (a cold and allergies) and just two symptoms (sneeze and fever), and let's assume that symptoms are deterministic.

```
var allergies = flip(0.3)
var cold = flip(0.2)

var sneeze = cold || allergies
var fever = cold

display([sneeze, fever])
```

Under this model, what is the probability that the patient is sneezing? What is the probability that the patient is sneezing *and* has a fever?

Question 2.4.2

Update the following code to inspect the joint probability distributions of **sneeze** and **fever** using **Infer**:

```
Infer(
  {method: "forward", samples: 1000},
  function() {
    ...
    return { ... }
  }
)
```

Question 2.4.3

If we wanted to represent the diseases of many patients we might have tried to make each disease and symptom into a function from a person to whether they have that disease, like this:

```

var allergies = function(person) {
  return flip(.3)
}
var cold = function(person) {
  return flip(.2)
}

var sneeze = function(person) {
  return cold(person) || allergies(person)
}

display([sneeze('bob'), sneeze('alice')])

```

Add **fever** to the program above, and use **Infer** to inspect the probability distribution over Bob's symptoms. Is this the same probability distribution that you computed for the single patient in Question 2.4.1? If not, what can you do to fix this program to capture our intuitions correctly?

Exercise 2.5

In this exercise you will work through the evaluation process for the **bend** higher-order function in this example:

```

var makeCoin = function(weight) {
  return function() {
    return flip(weight) ? 'h' : 't'
  }
}
var bend = function(coin) {
  return function() {
    return coin() == 'h'
      ? makeCoin(.7)()
      : makeCoin(.1)()
  }
}

var fairCoin = makeCoin(.5)
var bentCoin = bend(fairCoin)

```

Question 2.5.1

Directly compute the probability distribution of the bent coin in the example.

Question 2.5.2

Now, check your answer by using `Infer`.

```
// Your code here
```

Exercise 2.6

Question 2.6.1

Directly compute the probability that the geometric distribution defined by the following stochastic recursion returns the number 5.

Hint: What is the default parameter value for `flip()`?

```
var geometric = function() {  
  return flip() ? 0 : 1 + geometric()  
}
```

Question 2.6.2

Check your answer by using `Infer`.

Exercise 2.7

Quesiton 2.7.1

Convert the following probability table to a compact WebPPL program:

A	B	$\Pr(A, B)$
F	F	0.14
F	T	0.06

A	B	$\Pr(A, B)$
T	F	0.40
T	T	0.40

Requirement: fix the probability of A first and then define the probability of B to *depend* on whether A is true or not.

```
var A = ...
var B = ...
display([A, B])
```

Question 2.7.2

Run your WebPPL program and use **Infer** to check that you get the correct distribution.

Exercise 2.8

Below we've defined a higher-order function **flipSequence** that takes a coin flipping function (e.g. **trickCoin**, below) and flips that coin until it gets a *sequence* of two heads in a row (in which case it returns heads 'h') or two tails in a row (in which case it returns tails 't').

Try out different weights for the **trickCoin**:

```
var makeCoin = function(weight) {
  return function() {
    return flip(weight) ? 'h' : 't'
  }
}
var flipSequence = function(coin) {
  return function() {
    var flip1 = coin()
    var flip2 = coin()
    if (flip1 == flip2) {
      return flip1
    } else {
      return flipSequence(coin)()
    }
  }
}
```



```

    }
  }

  var trickCoin = makeCoin(.6)

  var n_samples = 10000;
  viz(
    Infer(
      {method: "forward", samples: n_samples},
      trickCoin
    )
  )
  viz(
    Infer(
      {method: "forward", samples: n_samples}, flipSequence(trickCoin)
    )
  )

```

Question 2.8.1

How does `flipSequence` change the distribution over return values (qualitatively)? Explain why requiring two flips in a row to be the same has this effect.

Question 2.8.2

What would happen if a fair coin (with weight 0.5) were input to `flipSequence`? Explain.

Exercise 2.9

Box2D is a two dimensional simulation engine for simulating rigid bodies (those with constant shape). It allows for the construction of arbitrary worlds and models important physical concepts including collisions, friction, gravity, momentum, and more.

Here we have provided a wrapper around Box2D that allows for the easy construction of worlds. A world consists of list of shapes. Shapes are created as JavaScript objects with the following properties:

shape	"circle" or "rect"
dims	[width, height] for rect or [radius] for circle
x	x position as number (distance from left)

y	y position as number (distance from top)
static	boolean (does the object move or stay still?)
velocity	[x_velocity, y_velocity]

The variables `worldWidth` and `worldHeight` are constants representing the visible size of the simulation window.

Here's an example with a ground and a single rectangle. Add another object to `bowlingWorld` and give it an initial velocity so that it knocks the original rectangle down:

```
var ground = {
  shape: 'rect',
  static: true,
  dims: [worldWidth, 10],
  x: worldWidth/2,
  y: worldHeight
}

var rect = {
  shape: 'rect',
  static: false,
  dims: [10, 100],
  x: worldWidth/2,
  y: 390
}

var bowlingWorld = [ground, rect]
physics.animate(1000, bowlingWorld)
```

Exercise 2.10

In [Example: Intuitive Physics](#), we modeled instability of a tower as the probability that the tower falls when perturbed, and we modeled “falling” as getting shorter. It would be reasonable to instead measure *how much shorter* the tower gets.

Question 2.10.1

Below, modify the stability/instability model by creating a continuous measure, `towerFallDegree`. Let this measure take different values between 0 and 1. That

way, your continuous measure will be numerically comparable to the discrete measure, `doesTowerFall` (defined here as either 0 or 1).

Explain what your continuous measure represents and why it might be a good continuous measure of instability.

```
///fold:
var listMin = function(xs) {
  if (xs.length == 1) {
    return xs[0]
  } else {
    return Math.min(
      xs[0],
      listMin(rest(xs))
    )
  }
}

var highestY = function (w) {
  listMin(
    map(
      function(obj) { return obj.y },
      w
    )
  )
}

const ground = {
  shape: 'rect',
  static: true,
  dims: [worldWidth, 10],
  x: worldWidth / 2,
  y: worldHeight + 6
}

const almostUnstableWorld = [
  ground,
  {shape: 'rect', static: false, dims: [24, 22], x: 175, y: 473},
  {shape: 'rect', static: false, dims: [15, 38], x: 159.97995044874122, y: 413},
  {shape: 'rect', static: false, dims: [11, 35], x: 166.91912737427202, y: 340},
  {shape: 'rect', static: false, dims: [11, 29], x: 177.26195677111082, y: 276},
  {shape: 'rect', static: false, dims: [11, 17], x: 168.51354470809122, y: 230}
]

var noisify = function (world) {
  var perturbX = function (obj) {
```

```

    var noiseWidth = 10
    if (obj.static) {
        return obj
    } else {
        return _.extend(
            {},
            obj,
            {
                x: uniform(
                    obj.x - noiseWidth,
                    obj.x + noiseWidth
                )
            }
        )
    }
}
map(perturbX, world)
}

///

// Returns height of tower
var getTowerHeight = function(world) {
    return worldHeight - highestY(world)
}

var doesTowerFall = function (initialW, finalW) {
    var approxEqual = function (a, b) {
        Math.abs(a - b) < 1.0
    }
    return 1 - approxEqual(
        highestY(initialW),
        highestY(finalW)
    )
}

var towerFallDegree = function(initialW, finalW) {
    ...
}

var visualizeInstabilityMeasure = function(measureFunction) {
    var initialWorld = noisify(almostUnstableWorld)

```

```

var finalWorld = physics.run(1000, initialWorld)
var measureValue = measureFunction(
  initialWorld,
  finalWorld
)
print("Instability measure: " + measureValue)
print("Initial height: " + getTowerHeight(initialWorld))
print("Final height: " + getTowerHeight(finalWorld))
physics.animate(1000, initialWorld)
}

// Test binary doesTowerFall measure
// visualizeInstabilityMeasure(doesTowerFall)

// Test custom towerFallDegree measure
visualizeInstabilityMeasure(towerFallDegree)

```

Question 2.10.2

Describe a tower with a very different `doesTowerFall` and `towerFallDegree` measures look like. Which measure captures the meaning of “unstable” better?

```

<script src='../assets/js/box2d.js'></script> <script src='../assets/js/physics.js'></script>
<script> document.addEventListener("DOMContentLoaded", () => document.querySelec-
torAll('pre').forEach((elt) => editor.setup(elt, language:'webppl')) ); </script>

```